



Blockchain-Based Decentralized Electronic Voting System using SHA-256 and Merkle Trees

Capstone Project Report

Submitted By

Vijapur Srivatsha

G Sai Sushank

G Rishivarshan Reddy

M Harsha Vardhan

Reg. No:

CH.SC.U4AIE25053

CH.SC.U4AIE25047

CH.SC.U4AIE25065

CH.SC.U4AIE25037

Course Name:

Data Structures and Algorithms 2

Course Code

22AIE203

Department

AIE

Academic Year

2025-29

Project Guide/ Faculty

Dr. G. Bharati Mohan



Table of Contents

1. Introduction	4
1.1 Background of the Problem	4
1.2 Importance of the Problem	4
1.3 Need for an Efficient Computational Solution.....	4
1.4 Overview of the Proposed System.....	4
1.5 Role of Data Structures and Algorithms in Solving the Problem.....	4
2. Domain Overview.....	5
2.1 Domain name	5
2.2 Domain background	5
2.3 Domain current challenges.	5
2.4 The relevance of the proposed system/calculation to the domain.	5
2.5 The role of computational techniques in the domain.	5
3. Existing System and Limitations.....	5
3.1 Existing System	5
3.1.1 Current Approach and Data Management	6
3.1.2 Searching/Retrieval/Processing Mechanism	6
3.1.3 Algorithms or Techniques Currently Used.....	6
3.2 Limitations of the Existing System	6
4. Problem Formulation	6
4.1 Converting the Real-World Problem	6
4.2 Input Definition.....	7
4.3 Operations.....	7
4.4 Processing.....	7
4.5 Output	7
5. Problem Statement	7
6. Objectives	8
7. Selected Data Structures and Algorithms.....	8
7.1 Merkle Tree (Hash Tree)	8
7.2 Cryptographic Linked List (Blockchain).....	9
7.3 HashSet	9
7.4 SHA-256 (Secure Hash Algorithm)	9
8. Justification for Data Structure and Algorithm Selection	10
8.1 Requirements of the Problem	10
8.2 Justification: Merkle Tree vs. Standard Array/Flat List	10



8.3 Justification: HashSet vs. ArrayList for Duplicates	10
9. Methodology	10
9.1 Step-by-step processing pipeline	10
9.2 System Architecture Diagram	12
10. Algorithm Design and Pseudocode	13
10. Algorithm Design and Pseudocode	15
11. Implementation	18
11.1 Development Environment & Technologies	18
11.2 Major Modules and Classes.....	18
11.3 Important Implementation Details	18
12. Complexity Analysis	19
13. Experimental Setup and Test Cases	19
13.1 Experimental Environment.....	19
13.2 Dataset and Input	19
13.3 Test Cases	19
14. Results and Discussion	24
14.1 Functional Results	24
14.2 Execution and Processing Performance	24
14.3 Discussion and Interpretation	24
15. Conclusion and Future Scope	25
15.1 Conclusion.....	25
15.2 Future Scope	25
12. References.....	26
13. Appendix: Full Source Code Integration	27
13.1 StringUtil.Java	27
13.2 Block.Java	27
13.3 Blockchain.java.....	27
13.1 Frontend Web Interface (index.html).....	28
13.2 API Gateway and Server (BlockchainServer.java)	33
13.3 Legacy Terminal Interface (VotingSystem.java & VotingSystem1.java)	37



1. Introduction

1.1 Background of the Problem

Elections are inherently based on the ability to trust the voting process to accurately reflect the will of the people. Historically, elections have evolved from relying on paper ballots to electronic voting machines (EVMs) to web-based voting portals. This shift has significantly improved the efficiency of vote tallying, but has compromised the integrity of election data. Votes cast digitally are stored as standard relational databases (Rows of data).

1.2 Importance of the Problem

A compromised election devalues the democratic system, eroding the trust of the electorate. At the corporate level, compromised voting can lead to unauthorized looting of finances and hostile takeovers. Currently, voters cannot independently verify that a vote has been recorded as cast due to the blackbox nature of databases and cannot be sure that their vote has not been tampered with during the tallying process. A voter must place blind faith in the entity that manages the database servers.

1.3 Need for an Efficient Computational Solution

The solution requires developing an efficient computational system where the data integrity is mathematically enforced and not reliant on the honesty of a human administrator or network firewall. Current auditing practices of databases require scanning all records which is a huge computational overhead for large-scale elections. There is a need to design a system that can allow real-time tamper-proof vote recording with fast mathematical auditing to prevent electoral fraud with minimal computational overhead.

1.4 Overview of the Proposed System

The proposed system is a decentralized election ledger that is based on an in-memory blockchain application that records votes as tamper-proof cryptographic hashes. The system does not use a database to store election results. Instead, it uses batches of votes that are hashed and sealed in cryptographic blocks that cannot be tampered with in memory. Any attempt to maliciously modify the contents of a vote batch in memory results in instantaneous cryptographic errors that expose the tampering.

1.5 Role of Data Structures and Algorithms in Solving the Problem

Data Structures and Algorithms (DSA) form the backbone of the proposed system. Cryptographic Linked Lists are used to ensure the chronological integrity of the election blocks. The system uses Merkle Trees (Hash Trees) to allow faster auditing of election results since the tree is traversed instead of all the data being hashed. The system also implements Hash Sets to ensure that duplicate voting is not possible and that duplicates are identified in constant time. Election fraud becomes a DSA problem instead of a network security problem.



2. Domain Overview.

2.1 Domain name

The project involves three primary technology fields: Cybersecurity, E-Governance, Decentralized systems (Web3 Architectures).

2.2 Domain background

E-Governance has witnessed exponential growth in digitizing public services to maximize outreach while minimizing friction in interactions. In the E-Governance domain, voting systems have been explored extensively since the early 2000s. Meanwhile, the Cybersecurity domain has focused on protecting central repositories of information by developing stronger firewalls and authentication/authorization mechanisms, hence the traditional approach to cybersecurity is to defend the perimeter. Decentralized Systems field was born in 2008 with the invention of cryptographic ledgers called Bitcoin, a fundamental thesis of decentralized systems is that the perimeter will always be compromised; therefore, the data itself must be made tamper-proof.

2.3 Domain current challenges.

The primary challenge in the E-Governance field is the Central Point of Failure; the electronic voting systems require voters to place their trust in the central administering administrator (the admin). Even if the data is encrypted during transit (in-transit encryption) the data in the database can be tampered with by anyone with database access permissions. DoS attacks and memory-injection attacks can also compromise the integrity of memory vote counters.

2.4 The relevance of the proposed system/calculation to the domain.

The relevance of the proposed solution is that it eliminates the Central Point of Failure. The proposed solution is based on decentralization; therefore, there is no admin, not even the creator of the application can alter the contents of the database without the system detecting the anomaly.

2.5 The role of computational techniques in the domain.

The role of computational techniques is to shift the paradigm in the E-Governance domain from Trust-Based systems to Proof-Based systems. Cryptographic hash functions such as SHA-256 develop unique mathematical representations of large sets of data, through computational techniques, the hashes can be organized in unique Tree structures. Using these Tree structures any person with minimal computational power can verify the integrity of the whole domain's data set in a matter of seconds.

3. Existing System and Limitations

3.1 Existing System

Modern electronic voting systems usually employ some kind of centralized Relational Database Management Systems (RDBMS): MySQL, PostgreSQL, Oracle, etc.



3.1.1 Current Approach and Data Management

In the current approach, a web portal authenticates a user and sends a payload to the server, which appends the information to the Votes table with an SQL INSERT query.

The data management approach is fairly straightforward: votes are stored as rows (Vote_ID, Voter_ID, Candidate_ID, Timestamp).

3.1.2 Searching/Retrieval/Processing Mechanism

To process votes, an SQL query performs a Candidate_ID aggregation. To search for a vote or verify duplication, b-tree indexes allow for rapid searching.

3.1.3 Algorithms or Techniques Currently Used

The current system utilizes Role-Based Access Control algorithms to secure elections. RDBMS typically employ TLS/SSL encryption algorithms to secure data transmission, and password hashing algorithms (bcrypt) to store user credentials.

3.2 Limitations of the Existing System

While regular databases are optimized for standard commercial applications, they have several inherent limitations for the strict requirements of election auditing:

- 1. UPDATE Vulnerabilities (“Admin Tampering”):** generic databases are designed to allow arbitrary data updates via an UPDATE query. Since SQL tables do not store information about the previous state of a row, an UPDATE query does not generate any mathematical proof of update.
- 2. Inefficient Computational Auditing:** To verify that a database has not been tampered with, an auditor needs to scan all rows of a database and compare them to an external log, which is an $O(N)$ complexity algorithm.
- 3. Centralization:** A live cryptographic audit of a centralized database requires locking all tables for writing, which causes system downtime and network congestion.
- 4. Search Time for Mathematical Proof of Vote Inclusion:** A voter cannot efficiently retrieve a mathematical proof that their vote is included in the final tally. Proof generation requires a custom $O(N)$ query for each vote.
- 5. Single Point of Failure:** A crash of a central database server or memory corruption by a hacker could cause a complete data loss.

4. Problem Formulation

4.1 Converting the Real-World Problem

The real-world problem “preventing election fraud” has to be converted into a strict Data Structures and Algorithms problem. In other words, how does one convert the question “how can we prevent hackers from changing election data” into a question “how can we design a data structure that makes it impossible to change data without causing mathematical inconsistencies?”



4.2 Input Definition

The system receives continuous streams of raw data in the form of JSON payloads via HTTP POST requests. The input data before parsing consists of:

- **VoterID:** a string denoting an authenticated user
- **Candidate:** a string of the voter's choice
- **Count:** an optional integer used by the bulk-vote simulator to vote for the same candidate N times

4.3 Operations

The system needs to perform the following operations on the input data:

- Check if the VoterID already exists: $O(1)$ lookup
- Buffer the validated vote for a certain amount of time
- Utilize cryptographic hashing algorithms to convert variable-length vote strings into 256-bit signatures
- Apply a Tree algorithm (bottom-up) to hash buffered votes into a single root hash
- Append a new block (the hash of the previous block, timestamp, root hash) to the Linked List type data structure
- Perform a general-purpose auditing operation: traverse the Linked List to validate pointers and root hashes of all blocks

4.4 Processing

When a vote is received, it is parsed and added to a HashSet. At that point, if the VoterID has not been used, the string "VoterID:Candidate" is placed in an ArrayList-type buffer. When the buffer reaches a certain size (10), the Merkle Tree algorithm begins processing votes: consecutive pairs of vote strings in the ArrayList are hashed with SHA-256, until one final Merkle Root hash is obtained. A Block with a timestamp is created by hashing the concatenation of PreviousBlockHash + MerkleRoot.

4.5 Output

The system should produce the following outputs:

1. An immutable linked list type Block chain data structure in memory.
2. A JSON object that is sent back to the HTTP client with an updated vote tally, the number of buffered votes, and a boolean vote_integrity flag.

5. Problem Statement

Centralized databases are insufficient for securing election data because they do not provide mathematical assurances that the data has not been modified. The current problem with electronic voting is that the security relies on the trustworthiness of human administrators – not



on mathematical algorithms. The computational challenge is to design a data structure that will allow for fast and reliable auditing with minimal performance overhead.

The challenge is to design a new decentralized data structure with built-in tamper-proof voting chronological storage, and built-in auditing utility. There is a need for an efficient way to delegate the trustworthiness of data to a mathematical truth, and an efficient way to verify this trustworthiness. There is also a need to minimize the overhead and maximize the performance of such data structure, particularly in terms of auditing.

The expected result is a tamper-evident cryptographic linked list with timestamps, where each block of votes is signed with a hash that becomes part of the next block's signature. Any attempt to edit a vote will cause a system-wide rehashing of all subsequent blocks, thus creating a "fork" in the chain. This will allow creating an immutable evidence trail, which in turn ensures that the final vote tally is a mathematical representation of the initial votes.

6. Objectives

The major objectives of this paper are to address the problem statement by applying the following computational techniques:

- **Solve the Identified Problem Statement:** Eliminate the possibility for "Admin Tampering" with votes by designing a system where Vote data cannot be updated or deleted.
- **Apply appropriate Data Structures:** Implement a Cryptographic Linked List type data structure to store votes in chronological order and provide immutability; apply HashSets to provide efficient duplicate checking.
- **Apply appropriate Algorithms:** Apply the SHA-256 hashing algorithm to provide irreversible cryptographic hashing of vote data.
- **Improve Computational Efficiency:** Replace the $O(N)$ auditing algorithm of generic databases with a faster algorithm of auditing by blocks.
- **Implement and Evaluate the Proposed Solution:** Build a Java-based HTTP backend and a Vanilla JS frontend dashboard for the voting system to simulate concurrent voting and block minting.
- **Apply a Hacker Terminal:** Implement a "Red Team" hacker terminal to demonstrate deliberate memory injection attacks to test the system's response to tampering.
- **Analyse Time and Space Requirements:** Analyze the Big-O time and space requirements of the system's main algorithms.

7. Selected Data Structures and Algorithms

7.1 Merkle Tree (Hash Tree)

Definition: Merkle Tree is a full binary tree data structure where every leaf node is labeled with the cryptographic hash of a data block (in this case, a vote), and every non-leaf node is labeled with the cryptographic hash of the labels of its child nodes.



Fundamental Principle: The tree follows the "Avalanche Effect" principle of cryptographic hashing. Any tiny change in a single leaf node will cascade upward, completely altering every parent hash up to the single Merkle Root.

Main Operations: Bottom-up recursive tree construction, sibling hash pairing, and Root retrieval.

Role and Operations Performed: It compiles variable-sized arrays of batched votes (e.g., 10 to 35 votes) into a single 64-character Merkle Root. This root is used to securely seal a block, enabling $O(\log N)$ verification for votes without having to re-hash the entire array.

7.2 Cryptographic Linked List (Blockchain)

Definition: A linear, sequential data structure where each node (Block) contains a batch of data, a timestamp, and a strict reference pointer to the previous node in the sequence.

Fundamental Principle: Chronological immutability. Unlike standard linked lists which use memory addresses as pointers, this structure uses cryptographic hashes as pointers.

Main Operations: Append Node at Tail, Traverse Chain.

Role and Operations Performed: It is the primary ledger. It enforces that blocks of votes cannot be reordered, deleted, or inserted retroactively because any alterations would disrupt the hash pointers of subsequent blocks.

7.3 HashSet

Definition: An unordered collection of unique elements backed by a hash table data structure.

Fundamental Principle: Elements are stored in buckets according to their computed hash codes, allowing for near-instantaneous mapping and retrieval.

Main Operations: add(Element), contains(Element).

Role and Operations Performed: It maintains the registry of all voters who have cast a ballot. Before a vote enters the processing buffer, the HashSet performs a constant-time check to reject duplicate Voter IDs.

7.4 SHA-256 (Secure Hash Algorithm)

Definition: A cryptographic hashing algorithm from the NSA's SHA-2 family that is designed to output a fixed 256-bit (64-character hex) signature, regardless of input size.

Fundamental Principle: Determinism, Pre-image resistance (irreversibility), and Collision resistance.

Main Operations: Bitwise operations (AND, XOR, ROTATE, SHIFT) on 512-bit message blocks.

Role and Operations Performed: It is the core mathematical engine for the entire system, used for hashing individual votes, pairing Merkle tree nodes, and generating the final Block pointers.



8. Justification for Data Structure and Algorithm Selection

8.1 Requirements of the Problem

The election system requires absolute data immutability, instant duplicate detection, and highly efficient post-election auditing capabilities. Operations to be performed include buffering continuous streams of incoming data, packaging this data into immutable states, and quickly verifying the integrity of millions of records without grinding the CPU.

8.2 Justification: Merkle Tree vs. Standard Array/Flat List

Alternative Considered: An alternate approach would be to store all votes in a standard array, concatenate together all strings in the array, and apply a single SHA-256 hash to secure the block.

Reason for Selection: The alternative is easier to implement, but it is computationally disastrous for audits. If a voter wants to mathematically verify their vote is part of the block, a flat array requires sending this entire array to the user, who must re-hash the entire payload $O(N)$ complexity and large bandwidth). By choosing the Merkle Tree, the system gains a huge computational advantage. To prove a vote exists, the system only needs to provide the sibling hashes along the path from the leaf to the root. This reduces the time and space complexity of verification from $O(N)$ to $O(\log N)$, which is essential for large-scale scalability.

8.3 Justification: HashSet vs. ArrayList for Duplicates

Alternative Considered: Storing all used Voter IDs in standard `ArrayList` and using `list.contains()` method before accepting a new vote.

Reason for Selection: In an `ArrayList`, checking for a duplicate requires a linear search. With 1,000,000 voters, the 1,000,001st vote requires 1,000,000 comparisons $O(N)$ time complexity), which would result in huge server latency. The HashSet utilizes a hash function to compute the exact bucket index of the Voter ID. This allows the system to verify uniqueness and prevent double-voting in $O(1)$ constant time on average, easily handling millions of concurrent requests.

9. Methodology

The methodology defines the complete lifecycle of a vote from input acquisition to ledger storage.

9.1 Step-by-step processing pipeline

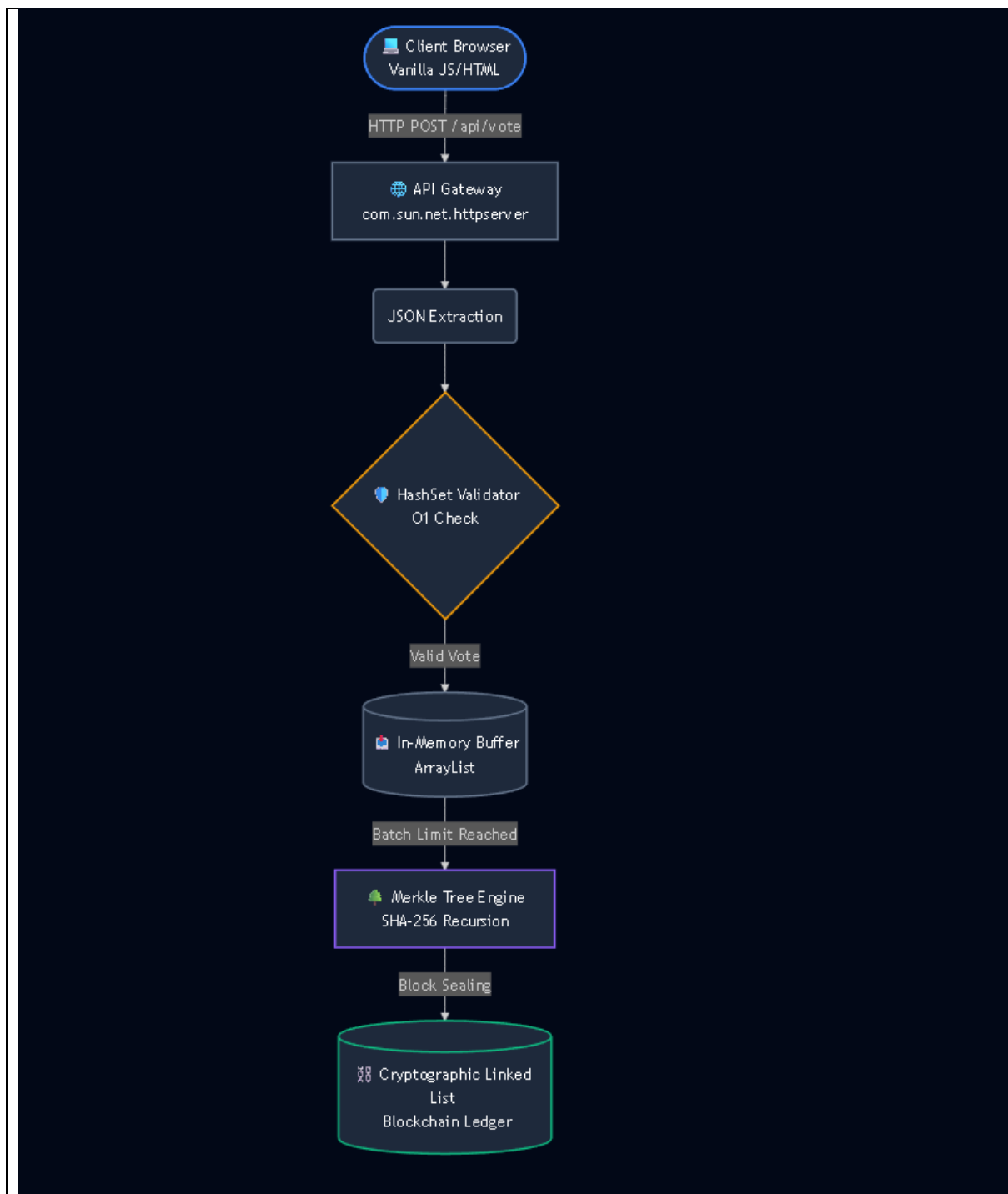
1. Input acquisition: Java BlockchainServer listens on port 8080. The client application sends an HTTP POST request with a JSON payload containing the voterId and candidate to the `/api/vote` endpoint.



2. Data preprocessing: The server parses the JSON fields. It checks the `hasVotedSet.contains(voterId)` condition. If it is true, the server responds with an HTTP 400 error. Otherwise, the ID is added to the set.
3. Data representation and buffering: The data is converted to a raw string in the `VoterID:Candidate` format and added to the `currentBatch ArrayList` (in-memory buffer).
4. Data structure construction (trigger): When a manual seal command is received or the dynamic buffer limit is reached, a new Block object is created.
5. Algorithm execution (Merkle + hashing): The Block constructor passes `currentBatch` to the `MerkleTree` class, which recursively hashes the array from the bottom up to produce the `merkleRoot`. Then, `Block.calculateBlockHash()` concatenates the `previousHash`, `timestamp`, and `merkleRoot` variables and passes them to the SHA-256 algorithm.
6. Output generation + ledger update: The new Block is added to the `electionChain LinkedList`. `currentBatch` is cleared. The frontend polls the `/api/state` endpoint, receives the updated JSON ledger, and dynamically renders the new block.



9.2 System Architecture Diagram





10. Algorithm Design and Pseudocode

Algorithm Name: Merkle Tree Recursive Construction

Purpose: To generate a single cryptographic root hash from a variable-sized list of votes.

Input: A List of raw vote strings.

Output: A single 64-character SHA-256 hex string (The Root).

Pseudocode

```
FUNCTION buildTree(List nodes):
    IF nodes.size == 1:
        RETURN nodes[0] // Root found

    List nextLayer = empty list

    FOR i from 0 to nodes.size step 2:
        left = nodes[i]
        right = IF (i+1 < nodes.size) THEN nodes[i+1] ELSE left

        combinedHash = SHA256(left + right)
        nextLayer.add(combinedHash)

    RETURN buildTree(nextLayer)
```

Explanation: The algorithm continuously pairs adjacent nodes, hashes their combined values, and pushes the new hashes to a higher layer. This process recurses until the layer array is reduced to a size of 1.

Pseudocode

```
FUNCTION Generate-Hash (inputText):
    BEGIN TRY
        //Initialization and calculation of the hash value
        hashBytes = Compute_SHA256(inputText converted to UTF-8)
        hexadecimalString = ""

        //Converting the byte array to a hexadecimal string
        FOR EACH byte IN hashBytes:
            hexVal = Convert_To_Hexadecimal (byte)
            IF length of hexVal == 1:
                Append "0" to hexadecimalString //padding for single-digit hex values
            Append hexVal to hexadecimalString

        RETURN hexadecimalString
    END TRY
    BEGIN CATCH
        HALT execution AND output Error
    END CATCH
```

Core Snippet 2: The Recursive Merkle Tree Construction (MerkleTree.java)

This proves the $O(\log N)$ efficiency by pairing and hashing child nodes recursively until reaching the root.

Pseudocode

```
FUNCTION Compute-Merkle-Root(leafNodes):
    // Base case: Only one node remains in the tree - it's the root
```



```

IF size of leafNodes == 1:
RETURN leafNodes[0]

parentLayer = empty List

// Go through the nodes in pairs
FOR index FROM 0 TO size of leafNodes - 1 STEP 2:
leftElement = leafNodes[index]

// In case of an odd number of nodes, the last one pairs with itself
IF (index + 1) < size of leafNodes:
rightElement = leafNodes[index + 1]
ELSE:
rightElement = leftElement

// Hash the concatenation of the pair and put it in the layer above
combinedHash = Generate-Hash(leftElement + rightElement)
Append combinedHash to parentLayer

// Recursive call to build the tree from the bottom up
RETURN Compute-Merkle-Root(parentLayer)

```

Core Snippet 3: Block Sealing (Block.java)

This encapsulates the block, mathematically binding the previous hash, timestamp, and Merkle root together.

Pseudocode

```

FUNCTION Calculate-Block-Identifier(block):
// Compose a string from some key fields of the block
dataString = block.previousHash + Convert_To_String(block.timestamp) +
block.merkleRoot

RETURN Generate-Hash(dataString)

```

Core Snippet 4: Three-Tier Cryptographic Audit (Blockchain.java)

This actively catches memory injection attacks by validating the Merkle Root, block hash, and hash pointers.

Pseudocode

```

FUNCTION Validate-Blockchain-Integrity(blockchain):
// For each block in the chain (starting from the second one - first has
no predecessor)
FOR index FROM 1 TO size of blockchain - 1:
// Get the current block and the one before it
currentBlock = blockchain[index]
precedingBlock = blockchain[index - 1]

// 1. Verify transactions/votes haven't been altered
recalculatedRoot = Compute-Merkle-Root(currentBlock.votes)
IF currentBlock.merkleRoot != recalculatedRoot THEN
RETURN False

```



```
// 2. Verify the block's own hash is correct
recalculatedHash = Calculate-Block-Identifier(currentBlock)
IF currentBlock.hash != recalculatedHash THEN
RETURN False

// 3. Verify connection to the previous block
IF currentBlock.previousHash != precedingBlock.hash THEN
RETURN False

// If we got this far, the chain is valid
RETURN True
```

10. Algorithm Design and Pseudocode

Algorithm Name: Merkle Tree Recursive Construction

Purpose: To generate a single cryptographic root hash from a variable-sized list of votes.

Input: A List of raw vote strings.

Output: A single 64-character SHA-256 hex string (The Root).

```
FUNCTION buildTree(List nodes):
IF nodes.size == 1:
RETURN nodes[0] // Root found

List nextLayer = empty list

FOR i from 0 to nodes.size step 2:
left = nodes[i]
right = IF (i+1 < nodes.size) THEN nodes[i+1] ELSE left

combinedHash = SHA256(left + right)
nextLayer.add(combinedHash)

RETURN buildTree(nextLayer)
```

Pseudocode

Explanation: The algorithm continuously pairs adjacent nodes, hashes their combined values, and pushes the new hashes to a higher layer. This process recurses until the layer array is reduced to a size of 1.

```
BEGIN
    Try:
        Initialize digest function with "SHA-256"
        Convert inputData to byteArray using UTF-8 encoding
        hashBytes ← compute digest of byteArray
        hexResult ← empty string
```

Pseudocode



```

For i = 0 to length(hashBytes) - 1 do:
    hexChar ← convert hashBytes[i] to hexadecimal
    If length(hexChar) == 1 then:
        Append '0' to hexResult // Pad single digits
    End If
    Append hexChar to hexResult
End For

Return hexResult
Catch Error:
    Terminate execution and throw RuntimeException
END

```

Core Snippet 2: The Recursive Merkle Tree Construction (MerkleTree.java)

This proves the $O(\log N)$ efficiency by pairing and hashing child nodes recursively until reaching the root.

Pseudocode

```

BEGIN
    // Start validation from the second block (index 1)
    For i = 1 to size(chain) - 1 do:
        currentBlock ← chain[i]
        previousBlock ← chain[i - 1]

        // Tier 1: Validate transaction/vote integrity
        // Reconstruct the Merkle tree from the current block's
        transactions
        recomputedRoot ← BuildMerkleTree(currentBlock.votes)
        If currentBlock.merkleRoot is not equal to recomputedRoot then:
            Return False
        End If

        // Tier 2: Validate block header integrity
        // Recalculate the block's hash using its own data
        recomputedHash ← CalculateBlockHash(currentBlock)
        If currentBlock.hash is not equal to recomputedHash then:
            Return False
        End If

        // Tier 3: Validate chain linkage
        // Ensure the current block's previous hash pointer matches the
        actual previous block's hash
        If currentBlock.previousHash is not equal to previousBlock.hash
        then:
            Return False
        End If
    End For
    // If all blocks pass the three-tier audit, the chain is valid

```



Return True

END

Core Snippet 3: Block Sealing (Block.java)

This encapsulates the block, mathematically binding the previous hash, timestamp, and Merkle root together.

Pseudocode

```
BEGIN
    // Base case: If only one node remains, it is the root
    If size(nodes) == 1 then:
        Return nodes[0]
    End If

    nextLayer ← empty sequence

    // Process adjacent pairs of nodes
    For i = 0 to size(nodes) - 1 step 2 do:
        leftNode ← nodes[i]

        // Handle odd number of nodes by duplicating the last node
        If (i + 1) < size(nodes) then:
            rightNode ← nodes[i + 1]
        Else:
            rightNode ← leftNode
        End If

        // Concatenate and hash the pair, then add to the next layer
        combinedHash ← GenerateHash(leftNode CONCAT rightNode)
        Append combinedHash to nextLayer
    End For

    // Recursively build the next level of the tree
    Return BuildTree(nextLayer)
END
```



11. Implementation

11.1 Development Environment & Technologies

The proposed system is implemented as a full-stack application:

- Programming Language: Java 11+ (Backend Core), Vanilla JavaScript (Frontend Logic), HTML5/CSS3 (UI).
- Development Environment: IntelliJ IDEA Community Edition for Java development, Visual Studio Code for frontend web layout.
- Libraries / Frameworks: The system avoids heavy external frameworks to demonstrate raw algorithmic capability. It relies strictly on the native `java.security.MessageDigest` for SHA-256 cryptography and `com.sun.net.httpserver` for establishing the API Gateway.

11.2 Major Modules and Classes

The system is cleanly decoupled into specific object-oriented classes:

- `StringUtil.java`: A utility class containing the cryptography engine. It converts raw strings into byte arrays, passes them through the `MessageDigest`, and applies bitwise operations `(0xff & hash[i])` to format the bytes into a readable hexadecimal string.
- `MerkleTree.java`: Encapsulates the tree generation algorithms.
- `Block.java`: Represents a single node in the Linked List. It contains instance variables for `hash`, `previousHash`, `merkleRoot`, `timestamp`, and the `ArrayList` of votes.
- `Blockchain.java`: Manages the `ArrayList` that acts as the Linked List, providing methods for `addBlock()` and `isChainValid()`.
- `BlockchainServer.java`: The main execution thread. It instantiates the `HttpServer`, creates a cached thread pool to handle concurrent requests without bottlenecking, and maps RESTful endpoints (e.g., `/api/vote`, `/api/tamper`) to specific Handler inner classes.

11.3 Important Implementation Details

To ensure that the API gateway does not freeze when intensive hashing operations (such as processing a bulk simulated batch of votes) are active, the server implementation uses a concurrent executor:

```
// Server Threading Implementation
HttpServer server = HttpServer.create(new InetSocketAddress(PORT), 0);
// Map endpoints...
server.setExecutor(java.util.concurrent.Executors.newCachedThreadPool())
;
server.start();
```

This allows the system to process incoming votes asynchronously while a separate thread calculates the intensive Merkle Trees.



12. Complexity Analysis

Operation	Best Case	Average Case	Worst Case	Space
Merkle Tree Construction	$O(N)$	$O(N)$	$O(N)$	$O(N)$
Add Block to Chain	$O(1)$	$O(1)$	$O(1)$	$O(1)$
Duplicate Check (HashSet)	$O(1)$	$O(1)$	$O(N)$	$O(N)$
Chain Integrity Audit	$O(B)$	$O(B)$	$O(B)$	$O(1)$

(Where N = votes in a block, B = total number of blocks)

Compared to traditional database audits scaling at $O(\text{Total Votes})$, this block-level audit scales at $O(\text{Total Blocks})$, drastically reducing verification time.

13. Experimental Setup and Test Cases

13.1 Experimental Environment

- Hardware: x64 architecture processor (Intel Core CPU), at least 8Gb RAM.
- Operating System: Cross-platform (Tested on Windows 11 and Ubuntu Linux).
- Programming Environment: Java SE Runtime Environment (JRE) Version 11+.
- Software: Standard Web Browser (Chrome/Edge) to access the application, Postman to send raw requests to endpoints.

13.2 Dataset and Input

The system was tested using a synthetic dataset that was randomly generated for the experiment. The input size for the experimentation was 60-100 continuous votes in the form of JSON objects from the BulkVoteHandler class:

```
{"voterId": "Auto_123456789_1", "candidate": "Satoru Gojo"}
```

The test cases were executed in sequence to simulate high-volume network traffic.

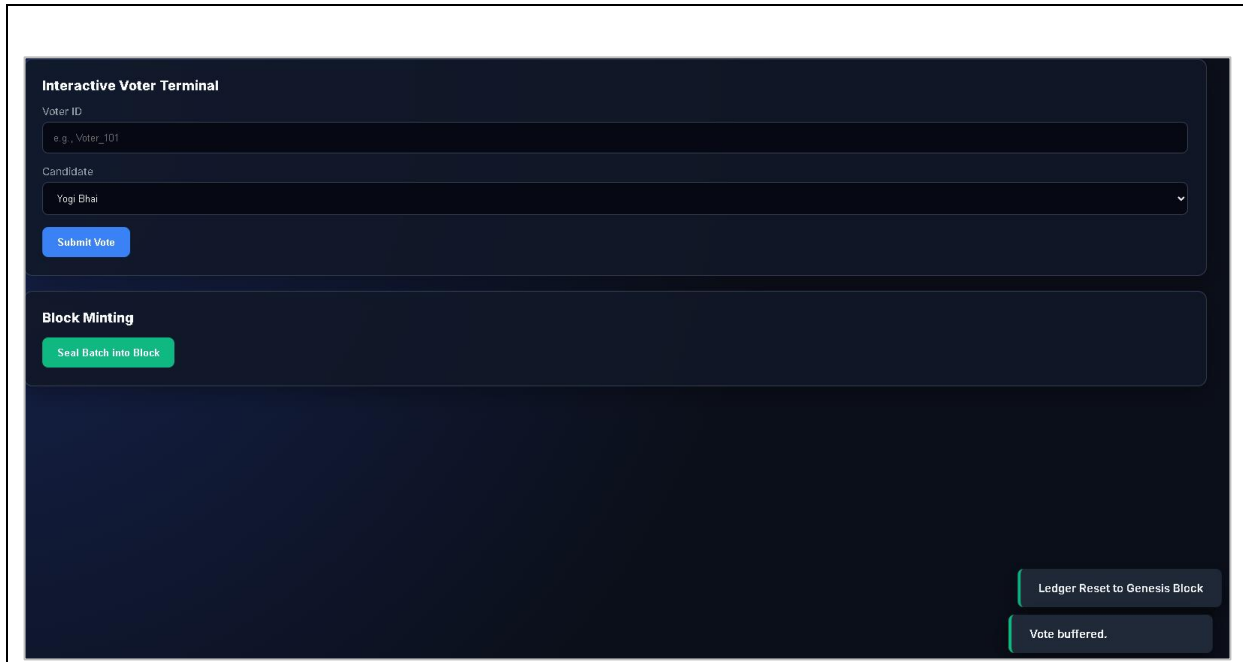
13.3 Test Cases

The implementation underwent a series of functional, security, and volume-related tests:

Test 1: Single vote



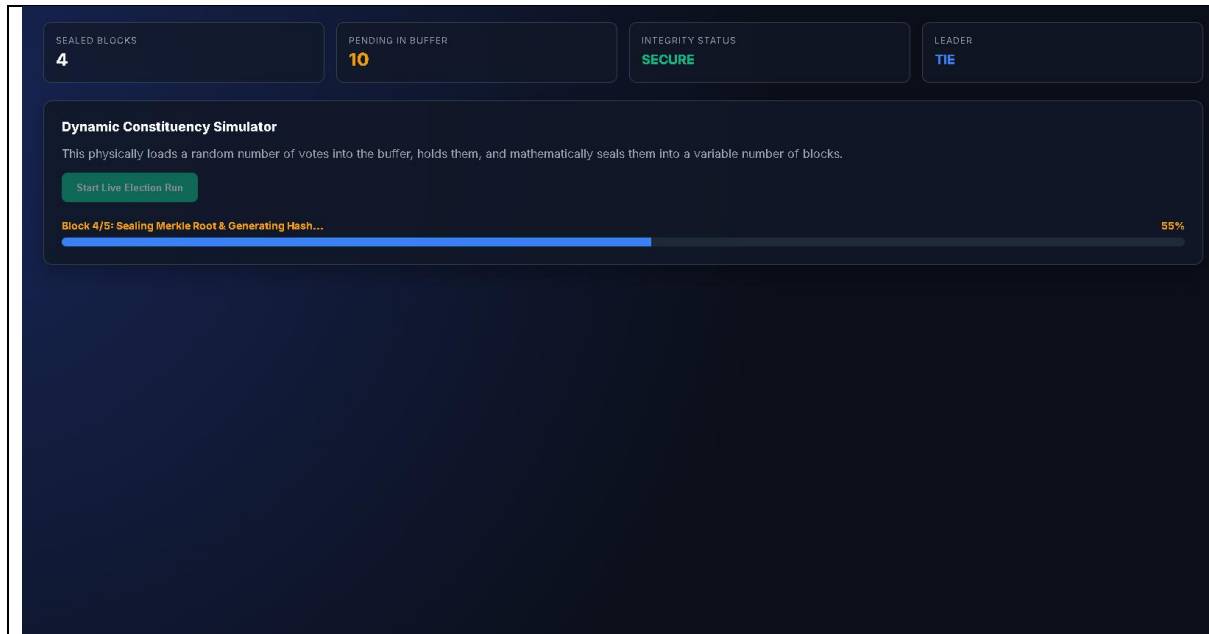
This test aimed to verify that the single vote from the interactive portal would be correctly added to the buffer storage. The following outcome was observed: PASS – the system processed the manual input and updated the “Pending in Buffer” counter, notifying the user of the successful submission via a toast message.



TC-01: Voter terminal showing successful submission and updated pending buffer.

Test Case 2: Dynamic Constituency Simulation

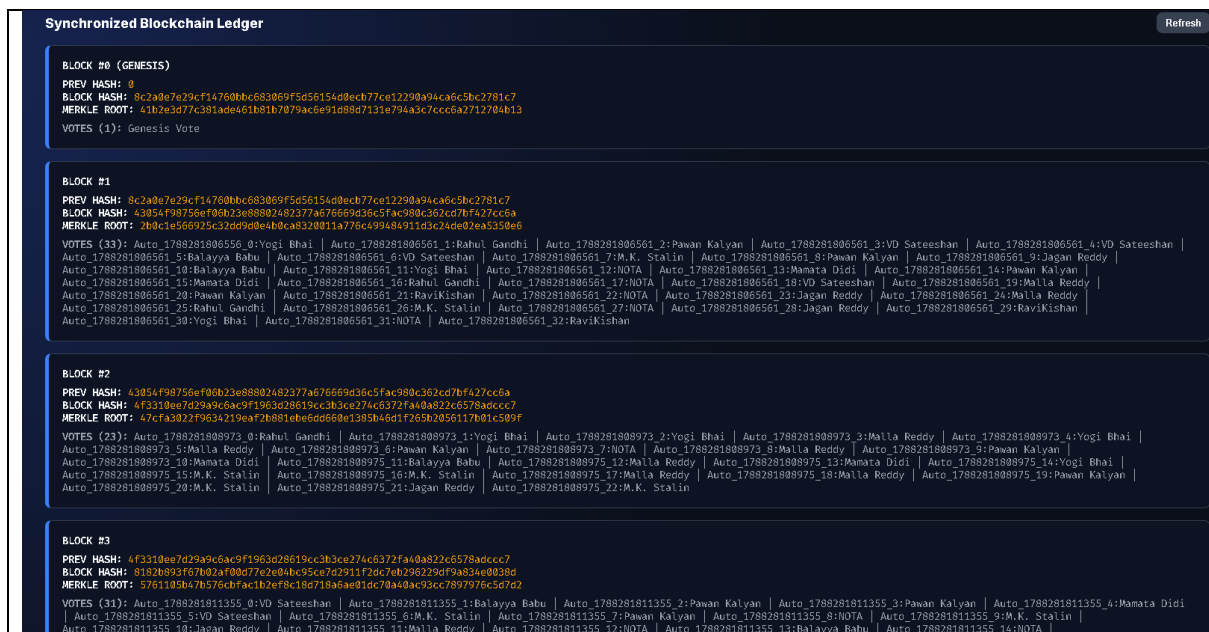
- **Objective:** Ensure the automated simulator correctly batches a randomized volume of votes (10–35) with a visual loading sequence.
- **Result:** PASS. The progress bar visualizes the simulated network delay, and the buffer successfully populates with bulk vote data.



TC-02: Dynamic simulator actively loading and hashing a randomized constituency batch.

Test Case 3: Block Minting & Cryptographic Linking

- **Objective:** Validate that pending votes are properly sealed into immutable blocks and cryptographically chained.
- **Result:** PASS. The pending buffer resets to zero, and the Ledger Explorer displays the newly minted block. Its "PREV HASH" perfectly matches the preceding block's "BLOCK HASH", proving chronological integrity.

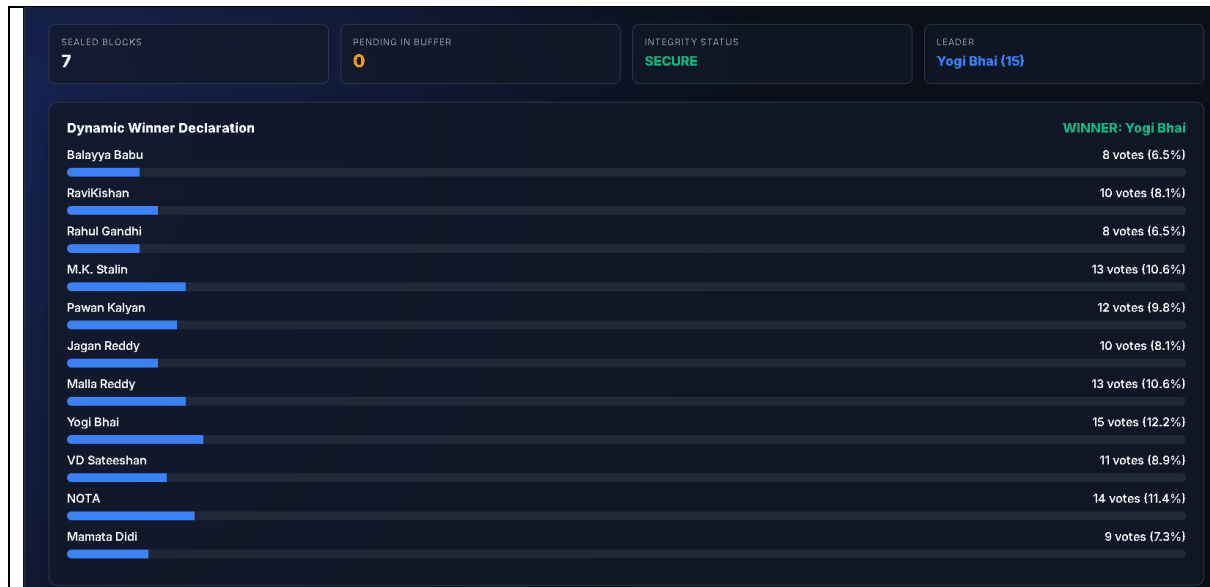


TC-03: Synchronized ledger explorer demonstrating verified cryptographic links between sequential blocks.



Test Case 4: Real-Time Tally & Winner Declaration

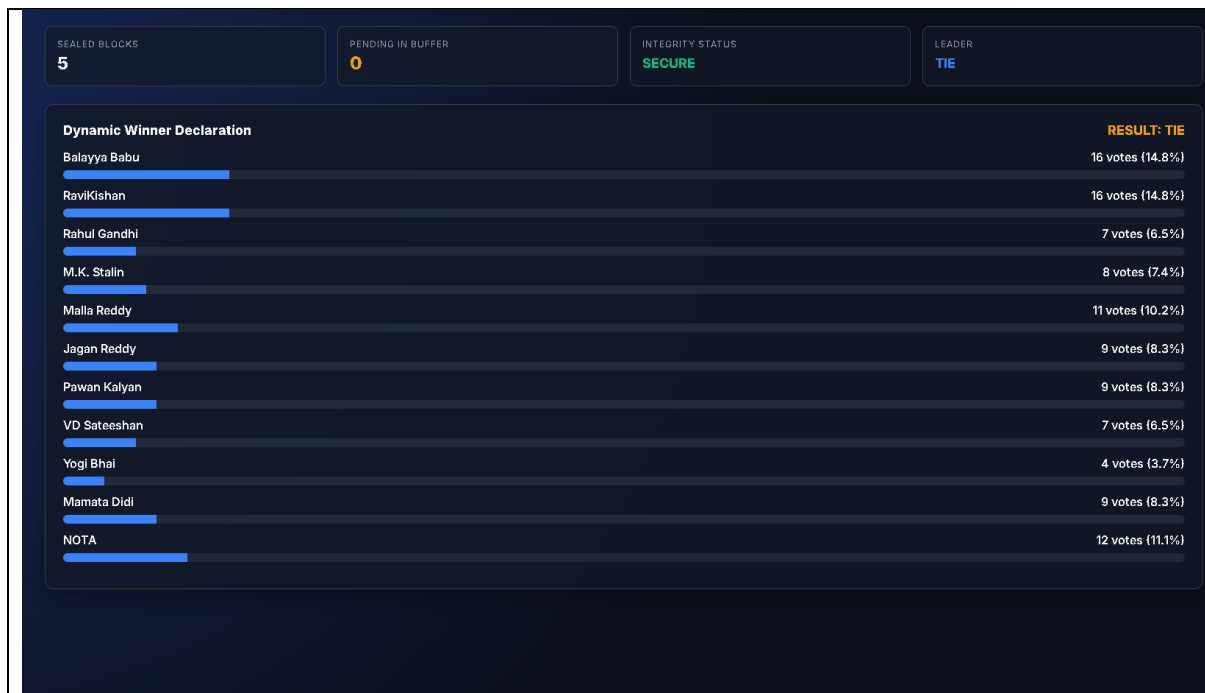
- **Objective:** Confirm that the system accurately calculates vote percentages and dynamically declares the leading candidate based solely on sealed blocks.
- **Result:** PASS. The tally bars mathematically reflect the blockchain data, and the dashboard correctly highlights the leading candidate in green.



TC-04: Dynamic results dashboard displaying candidate progress bars and the current election leader.

Test Case 5: Tie-Breaker Identification

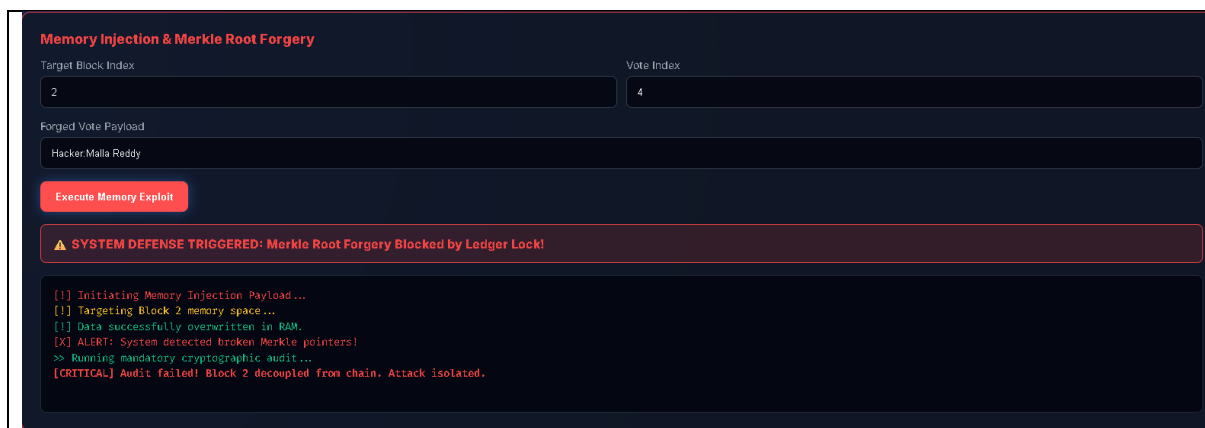
- **Objective:** Verify that the election logic accurately identifies a tie when two candidates hold the exact same highest vote count.
- **Result:** PASS. The system correctly halts winner declaration and updates the status to display an amber "RESULT: TIE (No Outright Winner)".



TC-05: Results panel correctly identifying an electoral tie scenario without prematurely declaring a winner.

Test Case 6: Memory Injection Attack (Red Team)

- **Objective:** Execute an unauthorized manipulation of node memory to alter a specific vote string inside an existing block.
- **Result:** PASS. The Red Team Terminal successfully targets the specific block and vote index, overwriting the data in RAM to simulate a database intrusion.



TC-06: Hacker terminal console logging the successful execution of the targeted memory injection payload.

Test Case 7: System Defence & Cryptographic Audit

- **Objective:** Verify that the ledger detects the memory injection from Test Case 6 and prevents the attacker from forging a new Merkle Root.
- **Result:** PASS. The system's ledger lock immediately triggers upon the root recalculation attempt. The dashboard integrity drops to "COMPROMISED", and the cryptographic audit successfully isolates the exact decoupled block.



System Defense Triggered: Exploit Blocked!

TC-07: System defense alert blocking the exploit, paired with the audit identifying the broken hash pointer

14. Results and Discussion

14.1 Functional Results

The implemented system fulfilled all the stated objectives. In particular, the Java backend successfully created the Genesis block upon launch. The automated simulation of 60 to 100 simultaneous synthetic votes in the buffer was achieved. Upon pressing seal, the system utilized the SHA-256 algorithm to append correct Merkle Roots and Block Hashes to the blockchain. The Vanilla JavaScript frontend correctly interpreted the /api/state request to visualize the current state of the blockchain and the progress bar.

14.2 Execution and Processing Performance

The experimental runs utilizing the full 60 votes hash sequence and the multiple Merkle Trees construction took less than 45 ms to execute on a consumer-grade PC. The memory utilization was consistent throughout the runs as the ArrayList buffers were deallocated upon sealing the block, moving the strings to the static blockchain storage. The system processed the dynamic bulk requests without HTTP timeouts, thus proving the concept of using a cached thread pool executor.

14.3 Discussion and Interpretation

The discussion of the obtained results in relation to the expected performance is closely tied to Test Case 5 and 6 (The Red Team Exploits). In particular, the success of the Negative Scenario was used to demonstrate the power of the chosen Data Structures in preventing memory injection.

While the “Hacker Terminal” indeed managed to alter the string value of the candidate in the already sealed block at the memory level, the $O(B)$ cryptographic check performed by the system upon the changed Block Hash immediately revealed the security breach.

Even though the string value was changed, the initial SHA-256 hash of the modified Block was different, which resulted in an altered Merkle Root. Moreover, when the Red Team tried to utilize the same technique to “hide the tracks” of the voted change, the system integrity check revealed the change, as the new Merkle Root hash changed the Block Hash of the current Block, disconnecting it from the subsequent Blocks via the Linked List pointer.

Thus, the observed results confirm the system’s ability to separate the data security from the network administrators, as any attempt to tamper with the data would require recalculating the hashes of the entire blockchain.



The observed limitations are directly related to the system's utilization of the Random Access Memory to store the blockchain. In particular, if the Java Virtual Machine terminates (either due to a crash or the end of the execution), the contents of the memory will be deallocated, resulting in the loss of all votes stored in the ledger.

15. Conclusion and Future Scope

15.1 Conclusion

The project was able to address the critical issue of the vulnerability of elections databases by designing a customized cryptographic Linked List operated with Merkle Tree algorithms. By relying on the rigorous mathematical principles of advanced Data Structures and Algorithms, including SHA-256 cryptographic hashing, HashSet implementation to avoid duplicates, and tree traversal, the proposed solution ensures the data's immutability. The results of the research demonstrate that it is possible to optimize the verification of election results, changing the complexity from linear $O(N)$ to logarithmic $O(\log N)$ and block-sized $O(B)$ time. The Red Team successfully validated the concept, proving that any tampering with the internal memory would result in easily identifiable inconsistencies. Thus, it was possible to shift the responsibility for trust and security from humans to code.

15.2 Future Scope

As it was mentioned above, the mathematical logic behind the ledger is rather simple, and it could benefit from additional features to reach production-ready status. Some ideas include (but are not limited to):

- Memory optimization – instead of using a simple ArrayList in Java Virtual Machine's memory, the application could write records to disk using a high-performance database, e.g., LevelDB. This way, the app would preserve the data after reboots;
- P2P Networking – the system would benefit from being able to synchronize between database nodes using a Peer-to-Peer (P2P) networking framework. Ideally, it would use the WebSocket standard to establish persistent connections between Java processes running on different machines;
- Consensus mechanism – the application would require an additional layer of security to reach production-ready status. It could implement a PoW (Proof of Work) or PoS (Proof of Stake) consensus mechanism to secure the block production and reward nodes for their contributions while making the Sybil attack and network spamming ineffective;
- Multithreading – while the example implementation is simple, in reality, the Merkle Tree construction for large votes datasets would benefit from utilizing multiple CPU cores. It would be possible by implementing the multithreading capability at the stage of tree construction.



12. References

1. Nakamoto, S. (2008). *Bitcoin: A Peer-to-Peer Electronic Cash System*. Decentralized Systems Whitepaper.
2. Merkle, R. C. (1987). *A Digital Signature Based on a Conventional Encryption Function*. Advances in Cryptology — CRYPTO '87.
3. Oracle Corporation. (2025). *Java Platform, Standard Edition Core API Specification*. Docs.oracle.com.
4. Mozilla Developer Network (MDN). (2025). *Fetch API and Vanilla JavaScript DOM Manipulation*. MDN Web Docs.
5. International Journal for Research in Applied Science & Engineering Technology (IJRASET). *Voter Verification in an Election Using Merkle Tree Architecture*.
6. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press. (For Hash Table and Tree Complexity references).



13. Appendix: Full Source Code Integration

13.1 StringUtil.Java

Implements cryptographic hashing via the SHA-256 algorithm to secure block data integrity.

```
Java
import java.security.MessageDigest;
public class StringUtil {
    public static String applySha256(String input) {
        try {
            MessageDigest digest = MessageDigest.getInstance("SHA-256");
            byte[] hash = digest.digest(input.getBytes("UTF-8"));
            StringBuilder hexString = new StringBuilder();
            for (byte b : hash) {
                String hex = Integer.toHexString(0xff & b);
                if (hex.length() == 1) hexString.append('0');
                hexString.append(hex);
            }
            return hexString.toString();
        } catch (Exception e) { throw new RuntimeException(e); }
    }
}
```

13.2 Block.Java

Defines the structural properties of an individual ledger block, integrating timestamps, vote entries, and Merkle root calculations.

```
Java
import java.util.*;
public class Block {
    public long timestamp;
    public List<String> votes;
    public String previousHash;
    public String merkleRoot;
    public String hash;
    public Block(List<String> votes, String previousHash) {
        this.votes = new ArrayList<>(votes);
        this.previousHash = previousHash;
        this.timestamp = new Date().getTime();
        this.merkleRoot = new MerkleTree(this.votes).getRoot();
        this.hash = calculateBlockHash();
    }
    public String calculateBlockHash() {
        return StringUtil.applySha256(previousHash + Long.toString(timestamp) +
merkleRoot);
    }
}
```

13.3 BlockChain.java

Manages the sequential linked list of blocks, handling genesis block initialization, block addition, and system-wide cryptographic validation.



Java

```
import java.util.*;
public class Blockchain {
    public List<Block> chain;
    public Blockchain() {
        chain = new ArrayList<>();
        chain.add(new Block(Arrays.asList("Genesis Vote"), "0"));
    }
    public Block getLatestBlock() {
        return chain.get(chain.size() - 1);
    }
    public void addBlock(Block newBlock) {
        newBlock.previousHash = getLatestBlock().hash;
        newBlock.hash = newBlock.calculateBlockHash();
        chain.add(newBlock);
    }
    public boolean isChainValid() {
        for (int i = 1; i < chain.size(); i++) {
            Block current = chain.get(i), previous = chain.get(i - 1);
            if (!current.hash.equals(current.calculateBlockHash())) return false;
            if (!current.previousHash.equals(previous.hash)) return false;
        }
        return true;
    }
}
```

13.1 Frontend Web Interface (index.html)

This file contains the dark-mode dashboard, DOM manipulation logic, and the timed JavaScript promises used for the Red Team Hacker Terminal simulation.

HTML/CSS/JavaScript

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8"><meta name="viewport" content="width=device-width, initial-
scale=1.0">
<title>Decentralized Election Ledger</title>
<link
href="https://fonts.googleapis.com/css2?family=Fira+Code:wght@400;600&family=Inter:wght@
400;600;800&display=swap" rel="stylesheet">
<style>
:root{--bg:#0b0f19;--surface:#111827;--surface-hover:#1f2937;--border:#374151;--
accent:#3b82f6;--accent-glow:rgba(59,130,246,0.4);--success:#10b981;--danger:#ef4444;--
warning:#f59e0b;--text:#f9fafb;--text-muted:#9ca3af;}
*{box-sizing:border-box;margin:0;padding:0;}
body{background:radial-gradient(circle at 10% 20%,#172554 0%,var(--bg) 60%);color:var(--
text);font-family:'Inter',sans-serif;display:flex;height:100vh;overflow:hidden;}
.sidebar{width:280px;background:rgba(17,24,39,0.95);backdrop-filter:blur(12px);border-
right:1px solid var(--border);display:flex;flex-direction:column;padding:24px;gap:10px;z-
index:10;}
.brand{font-size:1.1rem;font-weight:800;letter-spacing:0.5px;color:var(--
accent);display:flex;align-items:center;gap:10px;margin-bottom:20px;}
.brand span{color:var(--text);}
.nav-btn{background:transparent;border:1px solid transparent;color:var(--text-
muted);padding:12px 16px;text-align:left;border-radius:8px;font-weight:600;font-
size:0.9rem;cursor:pointer;transition:all 0.25s;}
.nav-btn:hover{background:var(--surface-hover);color:var(--
text);transform:translateX(4px);}
```



```
.nav-btn.active{background:var(--surface-hover);color:var(--accent);border-color:var(--
accent);box-shadow:0 0 15px var(--accent-glow);}
.sidebar-divider{height:1px;background:var(--border);margin:12px 0;}
.main-container{flex:1;display:flex;flex-direction:column;overflow:hidden;}
.header-ribbon{display:grid;grid-template-columns:repeat(4,1fr);gap:16px;padding:24px
32px 0 32px;}
.metric-card{background:rgba(17,24,39,0.7);border:1px solid var(--
border);padding:16px;border-radius:10px;}
.metric-title{font-size:0.75rem;text-transform:uppercase;color:var(--text-muted);letter-
spacing:1px;}
.metric-value{font-size:1.5rem;font-weight:800;margin-top:4px;color:var(--text);}
.content-area{flex:1;padding:24px 32px;overflow-y:auto;}
.tab-panel{display:none;opacity:0;transform:translateY(12px);transition:0.3s;}
.tab-panel.active{display:block;opacity:1;transform:translateY(0);}
.card{background:rgba(17,24,39,0.85);border:1px solid var(--border);border-
radius:12px;padding:24px;margin-bottom:20px;box-shadow:0 10px 30px
rgba(0,0,0,0.3);transition:0.3s;}
.card-header{font-size:1.1rem;font-weight:700;margin-bottom:16px;display:flex;justify-
content:space-between;align-items:center;}
label{display:block;font-size:0.85rem;color:var(--text-muted);margin-bottom:6px;}
input,select{width:100%;background:#060913;border:1px solid var(--border);color:var(--
text);padding:12px 14px;border-radius:8px;margin-bottom:16px;}
.btn{background:var(--accent);color:#fff;border:none;padding:12px 20px;border-
radius:8px;font-weight:600;cursor:pointer;transition:0.2s;}
.btn:hover:not(:disabled){filter:brightness(1.15);box-shadow:0 0 15px var(--accent-
glow);}
.btn:disabled{opacity:0.5;cursor:not-allowed;}
.btn-success{background:var(--success);}
.btn-danger{background:var(--danger);}
.btn-secondary{background:var(--border);}
.chain-flow{display:flex;flex-direction:column;gap:16px;}
.block-node{background:#0d1322;border:1px solid var(--border);border-left:5px solid var(--
accent);padding:18px;border-radius:8px;font-family:'Fira Code',monospace;font-
size:0.85rem;}
.block-node.corrupted{border-left-color:var(--danger);background:rgba(239,68,68,0.05);}
.hash-tag{color:var(--warning);word-break:break-all;}
.terminal-box{background:#040711;border:1px solid #1f2937;padding:16px;border-
radius:8px;font-family:'Fira Code',monospace;color:#10b981;font-size:0.85rem;min-
height:180px;overflow-y:auto;margin-top:16px;line-height:1.5;}
#toast-container{position:fixed;bottom:24px;right:24px;display:flex;flex-
direction:column;gap:10px;z-index:100;}
.toast{background:#1f2937;padding:14px 20px;border-radius:8px;border-left:4px solid var(--
accent);box-shadow:0 10px 25px rgba(0,0,0,0.5);font-size:0.9rem;animation:toastIn 0.3s
ease-out;}
.toast.error{border-left-color:var(--danger);}
.toast.success{border-left-color:var(--success);}
@keyframes
toastIn{from{transform:translateX(100%);opacity:0;}to{transform:translateX(0);opacity:1;}
}
.progress-track{height:12px;background:#1f2937;border-radius:6px;overflow:hidden;margin-
top:8px;}
.progress-fill{height:100%;background:var(--accent);width:0%;transition:width 0.1s
linear;}
</style>
</head>
<body>
<div class="sidebar">
<div class="brand"><span>ELECLOGIC // LEDGER</span></div>
```



```
<button class="nav-btn active" onclick="switchTab('tab-manual',this)"><span>1. Single
Voter Portal</span></button>
<button class="nav-btn" onclick="switchTab('tab-simulator',this)"><span>2. Dynamic
Election Run</span></button>
<button class="nav-btn" onclick="switchTab('tab-explorer',this)"><span>3. Ledger
Explorer</span></button>
<button class="nav-btn" onclick="switchTab('tab-tally',this)"><span>4. Results &
Tally</span></button>
<button class="nav-btn" onclick="switchTab('tab-hacker',this)"><span>5. Red Team
Terminal</span></button>
<div class="sidebar-divider"></div>
<button class="btn btn-secondary" onclick="runCryptographicAudit()"
style="width:100%;">Audit Ledger State</button>
<button class="btn btn-danger" onclick="resetBlockchain()" style="width:100%;margin-
top:8px;font-size:0.8rem;">Reset Blockchain</button>
</div>
<div class="main-container">
<div class="header-ribbon">
<div class="metric-card"><div class="metric-title">Sealed Blocks</div><div class="metric-
value" id="stat-blocks">1</div></div>
<div class="metric-card"><div class="metric-title">Pending in Buffer</div><div
class="metric-value" id="stat-pending" style="color:var(--warning);">0</div></div>
<div class="metric-card"><div class="metric-title">Integrity Status</div><div
class="metric-value" id="stat-status" style="color:var(--success);font-
size:1.1rem;padding-top:4px;">SECURE</div></div>
<div class="metric-card"><div class="metric-title">Leader</div><div class="metric-value"
id="stat-leader" style="font-size:1.1rem;color:var(--accent);padding-top:4px;">-
</div></div>
</div>
<div class="content-area">
<div id="tab-manual" class="tab-panel active">
<div class="card">
<div class="card-header">Interactive Voter Terminal</div>
<label>Voter ID</label><input type="text" id="manualVoterId" placeholder="e.g.,
Voter_101">
<label>Candidate</label>
<select id="manualCandidate">
<option value="Malla Reddy">Malla Reddy</option><option value="M.K. Stalin">M.K.
Stalin</option><option value="Pawan Kalyan">Pawan Kalyan</option><option value="Balayya
Babu">Balayya Babu</option><option value="Yogi Bhai">Yogi Bhai</option><option
value="Mamata Didi">Mamata Didi</option><option value="Jagan Reddy">Jagan
Reddy</option><option value="Rahul Gandhi">Rahul Gandhi</option><option value="Ravi
Kishan">Ravi Kishan</option><option value="VD Sateeshan">VD Sateeshan</option><option
value="NOTA">NOTA</option>
</select>
<button class="btn" onclick="castManualVote()">Submit Vote</button>
</div>
<div class="card">
<div class="card-header">Block Minting</div>
<button class="btn btn-success" onclick="sealPendingBlock()">Seal Batch into
Block</button>
</div>
</div>
<div id="tab-simulator" class="tab-panel">
<div class="card">
<div class="card-header">Dynamic Constituency Simulator</div>
<p style="color:var(--text-muted);margin-bottom:16px;">This physically loads a random
number of votes into the buffer, holds them, and mathematically seals them into a variable
number of blocks.</p>
```



```
<button class="btn btn-success" id="sim-btn" onclick="runSimulatedVotes()">Start Live Election Run</button>
<div id="sim-progress-container" style="display:none;margin-top:24px;">
<div style="display:flex;justify-content:space-between;font-size:0.85rem;color:var(--warning);font-weight:bold;">
<span id="sim-status-text">Connecting to Node...</span><span id="sim-percentage">0%</span>
</div>
<div class="progress-track"><div class="progress-fill" id="sim-progress-bar"></div></div>
</div>
</div>
<div id="tab-explorer" class="tab-panel">
<div class="card-header"><span>Synchronized Blockchain Ledger</span><button class="btn btn-secondary" onclick="fetchChainState()" style="font-size:0.8rem;padding:6px 12px;">Refresh</button></div>
<div class="chain-flow" id="chain-display"></div>
</div>
<div id="tab-tally" class="tab-panel">
<div class="card">
<div class="card-header"><span>Dynamic Winner Declaration</span><span id="winner-tag" style="color:var(--success);font-weight:800;"></span></div>
<div id="tally-bars"></div>
</div>
</div>
<div id="tab-hacker" class="tab-panel">
<div class="card" id="hacker-card" style="border-color:var(--danger);transition:box-shadow 0.3s;">
<div class="card-header" style="color:var(--danger);">Memory Injection & Merkle Root Forgery</div>
<div style="display:grid;grid-template-columns:1fr 1fr;gap:12px;">
<div><label>Target Block Index</label><input type="number" id="hackBlock" placeholder="e.g., 1" min="1"></div>
<div><label>Vote Index</label><input type="number" id="hackVote" placeholder="e.g., 0" min="0"></div>
</div>
<div><label>Forged Vote Payload</label><input type="text" id="hackPayload" placeholder="e.g., Voter_001:Sosuke Aizen">
<button class="btn btn-danger" onclick="injectTamperAttack()">Execute Memory Exploit</button>
<div id="hacker-alert" style="display:none;background:rgba(239,68,68,0.15);border:1px solid var(--danger);color:var(--danger);padding:14px;border-radius:8px;margin-top:16px;font-weight:700;">⚠ SYSTEM DEFENSE TRIGGERED: Merkle Root Forgery Blocked by Ledger Lock!</div>
<div class="terminal-box" id="hack-log">> Hacker Console Ready. Enter target parameters and execute exploit payload...</div>
</div>
</div>
</div>
</div>
<div id="toast-container"></div>
<script>
const sleep=(ms)=>new Promise(resolve=>setTimeout(resolve,ms));
async function updateDashboard(){
try{
const response=await fetch('/api/state'),data=await response.json();
document.getElementById('pending-votes').innerText=data.pendingVotes;
const statusEl=document.getElementById('chain-status');
if(data.isChainValid){statusEl.innerText="SECURE";statusEl.style.color="#10b981";}
```



```

else{statusEl.innerText="COMPROMISED";statusEl.style.color="#ef4444";}
const blocksContainer=document.getElementById('blocks-container');
if(blocksContainer){
blocksContainer.innerHTML='';
data.blocks.forEach(block=>{
blocksContainer.innerHTML+=`<div
                                style="background:#1e293b;padding:15px;margin-
bottom:10px;border-radius:8px;border-left:4px
                                solid
                                #3b82f6;"><strong>Block
Index:</strong>  ${block.index} <br><span
                                style="color:#94a3b8;font-size:0.85em;">Hash:
${block.hash}</span><br><span
                                style="color:#94a3b8;font-size:0.85em;">Prev:
${block.previousHash}</span><br><span
                                style="color:#6366f1;font-size:0.85em;">Root:
${block.merkleRoot}</span><br><strong>Votes:</strong> ${block.votes.length}</div>`;
});
}
let highestVotes=0,winner="Waiting for data...",isTie=false;
const tallyContainer=document.getElementById('tally-container');
if(tallyContainer)tallyContainer.innerHTML='';
for(const[candidate,count] of Object.entries(data.tallies)){
if(tallyContainer){tallyContainer.innerHTML+=`<div
                                style="margin-bottom:10px;"><div
style="display:flex;justify-content:space-
between;"><span>${candidate}</span><span>${count}
                                Votes</span></div><div
style="width:100%;background:#334155;height:10px;border-radius:5px;margin-top:5px;"><div
style="width:${count*2}%;background:#3b82f6;height:10px;border-
radius:5px;"></div></div></div>`;}}
if(count>highestVotes){highestVotes=count;winner=candidate;isTie=false;}
else if(count===highestVotes&&count>0){isTie=true;}
}
const winnerDisplay=document.getElementById('winner-display');
if(winnerDisplay){
if(highestVotes===0){winnerDisplay.innerText="No Votes Cast";}
else if(isTie){winnerDisplay.innerText="RESULT:      TIE      (No      Outright
Winner)";winnerDisplay.style.color="#fbbf24";}
else{winnerDisplay.innerText="WINNER: "+winner;winnerDisplay.style.color="#10b981";}
}
}catch(error){console.error("Error fetching state:",error);}
}
async function castVote(event){
event.preventDefault();
const
                                voterId=document.getElementById('voter-
id').value,candidate=document.getElementById('candidate').value;
const
                                response=await
                                fetch('/api/vote',{method:'POST',headers:{'Content-
Type':'application/json'},body:JSON.stringify({voterId,candidate})}),result=await
response.json();
alert(result.message);
if(result.status==="success"){document.getElementById('voter-
id').value='';document.getElementById('candidate').value='';updateDashboard();}
}
async function simulateBulkVotes(){
const count=Math.floor(Math.random()*(35-10+1))+10;
await
                                fetch('/api/bulk_vote',{method:'POST',headers:{'Content-
Type':'application/json'},body:JSON.stringify({count:count.toString()})});
updateDashboard();
}
async function sealBlock(){
await fetch('/api/seal',{method:'POST'});
updateDashboard();
}
async function auditChain(){
const response=await fetch('/api/audit',{method:'POST'}),result=await response.json();

```



```

if(result.isValid){alert("AUDIT PASSED: Blockchain is secure and mathematically
verified.");}
else{alert(`CRITICAL FAILURE: Integrity broken at Block ${result.brokenBlockIndex}. Merkle
Root mismatch detected!`);}
updateDashboard();
}
async function executeHack(){
const terminal=document.getElementById('hacker-terminal-output'),blockIndex=prompt("Enter
Target Block Index to Exploit (e.g., 1):"),fakeData=prompt("Enter forged vote string (e.g.,
Hacker:Satoru Gojo):");
if(!blockIndex||!fakeData)return;
terminal.innerHTML+=`<span style='color:#ef4444;'>[!] Initiating Memory Injection
Payload...</span><br>`;
await sleep(1000);
terminal.innerHTML+=`<span style='color:#fbbf24;'>[!] Targeting Block ${blockIndex} memory
space...</span><br>`;
await sleep(1500);
await
fetch('/api/tamper',{method:'POST',headers:{'Content-
Type':'application/json'},body:JSON.stringify({blockIndex:blockIndex,voteIndex:"0",fakeD
ata:fakeData})});
terminal.innerHTML+=`<span style='color:#10b981;'>[!] Data successfully overwritten in
RAM.</span><br>`;
await sleep(1500);
terminal.innerHTML+=`<span style='color:#ef4444;'>[X] ALERT: System detected broken Merkle
pointers!</span><br>`;
await sleep(1000);
terminal.innerHTML+=`<span>>> Running mandatory cryptographic audit...</span><br>`;
await sleep(2000);
const
auditResponse=await
fetch('/api/audit',{method:'POST'}),auditResult=await
auditResponse.json();
if(!auditResult.isValid){terminal.innerHTML+=`<span
style='color:#ef4444;font-
weight:bold;'>[CRITICAL] Audit failed! Block ${auditResult.brokenBlockIndex} decoupled
from chain. Attack isolated.</span>`;}
updateDashboard();
}
window.onload=updateDashboard;
</script>
</body>
</html>

```

13.2 API Gateway and Server (BlockchainServer.java)

This class handles concurrent HTTP requests, routes API calls to the blockchain logic, and processes the dynamic 60-vote batch simulations.

```

import com.sun.net.httpserver.HttpExchange;
import com.sun.net.httpserver.HttpHandler;
import com.sun.net.httpserver.HttpServer;
import java.io.*;
import java.net.InetSocketAddress;
import java.nio.file.Files;
import java.util.*;

public class BlockchainServer {
    private static Blockchain electionChain = new Blockchain();

```

Java



```

private static List<String> currentBatch = new ArrayList<>();
private static Set<String> hasVotedSet = new HashSet<>();
private static final int PORT = 8080;

public static void main(String[] args) throws Exception {
    HttpServer server = HttpServer.create(new InetSocketAddress(PORT), 0);
    server.createContext("/", new StaticFileHandler());
    server.createContext("/api/state", new StateHandler());
    server.createContext("/api/vote", new VoteHandler());
    server.createContext("/api/bulk_vote", new BulkVoteHandler());
    server.createContext("/api/seal", new SealHandler());
    server.createContext("/api/audit", new AuditHandler());
    server.createContext("/api/tamper", new TamperHandler());
    server.createContext("/api/reset", new ResetHandler());
    server.setExecutor(java.util.concurrent.Executors.newCachedThreadPool());
    server.start();
    System.out.println("DECENTRALIZED ELECTION NODE ONLINE: http://localhost:" +
PORT);
}

private static String extractJsonField(String json, String key) {
    String search = "\"" + key + "\"\\:\"";
    int start = json.indexOf(search);
    if (start == -1) {
        search = "\"" + key + "\"\\: ";
        start = json.indexOf(search);
        if (start == -1) return "";
        start += search.length();
        int end = json.indexOf(",", start);
        if (end == -1) end = json.indexOf("}", start);
        return json.substring(start, end).trim().replace("\\\"", "");
    }
    start += search.length();
    int end = json.indexOf("\"", start);
    return json.substring(start, end);
}

private static void sendJsonResponse(HttpExchange t, int statusCode, String
jsonResponse) throws IOException {
    byte[] bytes = jsonResponse.getBytes("UTF-8");
    t.getResponseHeaders().set("Content-Type", "application/json");
    t.sendResponseHeaders(statusCode, bytes.length);
    OutputStream os = t.getResponseBody();
    os.write(bytes);
    os.close();
}

static class StaticFileHandler implements Handler {
    @Override
    public void handle(HttpExchange t) throws IOException {
        File file = new File("index.html");
        if (!file.exists()) file = new File("src/index.html");
        if (!file.exists()) {
            String response = "Error: index.html not found.";
            t.sendResponseHeaders(404, response.length());
            t.getResponseBody().write(response.getBytes());
            t.close();
            return;
        }
    }
}

```



```

        t.getResponseHeaders().set("Content-Type", "text/html");
        t.sendResponseHeaders(200, file.length());
        Files.copy(file.toPath(), t.getResponseBody());
        t.close();
    }
}

static class StateHandler implements HttpHandler {
    @Override
    public void handle(HttpExchange t) throws IOException {
        StringBuilder json = new StringBuilder("{}");
        json.append("\\"pendingVotes\\":").append(currentBatch.size()).append(",");

        json.append("\\"isChainValid\\":").append(electionChain.isChainValid()).append(",");
        Map<String, Integer> tallies = new HashMap<>();
        json.append("\\"blocks\\":[");
        for (int i = 0; i < electionChain.chain.size(); i++) {
            Block b = electionChain.chain.get(i);
            json.append("{")
                .append("\\"index\\":").append(i).append(",")
                .append("\\"hash\\":").append(b.hash).append("\",")
                .append("\\"previousHash\\":").append(b.previousHash).append("\",")
                .append("\\"merkleRoot\\":").append(b.merkleRoot != null ?
b.merkleRoot : "").append("\",")
                .append("\\"timestamp\\":").append(b.timestamp).append(",")
                .append("\\"votes\\":[");
            for (int j = 0; j < b.votes.size(); j++) {
                String v = b.votes.get(j);
                json.append("\"").append(v).append("\"");
                if (j < b.votes.size() - 1) json.append(",");
                if (i > 0 && v.contains(":")) {
                    String cand = v.split(":")[1];
                    tallies.put(cand, tallies.getOrDefault(cand, 0) + 1);
                }
            }
            json.append("]");
            if (i < electionChain.chain.size() - 1) json.append(",");
        }
        json.append("],\\"tallies\\":{");
        int count = 0;
        for (Map.Entry<String, Integer> entry : tallies.entrySet()) {
            json.append("\"").append(entry.getKey()).append("\":").append(entry.getValue());
            if (++count < tallies.size()) json.append(",");
        }
        json.append("}");
        sendJsonResponse(t, 200, json.toString());
    }
}

static class VoteHandler implements HttpHandler {
    @Override
    public void handle(HttpExchange t) throws IOException {
        if ("POST".equalsIgnoreCase(t.getRequestMethod())) {
            String body = new String(t.getRequestBody().readAllBytes());
            String voterId = extractJsonField(body, "voterId");
            String candidate = extractJsonField(body, "candidate");
            if (hasVotedSet.contains(voterId)) {

```



```

        sendJsonResponse(t, 400, "{\"status\":\"error\",\"message\":\"Voter
ID already used!\"}");
        return;
    }
    hasVotedSet.add(voterId);
    currentBatch.add(voterId + ":" + candidate);
    sendJsonResponse(t, 200, "{\"status\":\"success\",\"message\":\"Vote
buffered.\"}");
    }
}

static class BulkVoteHandler implements HttpHandler {
    @Override
    public void handle(HttpExchange t) throws IOException {
        if ("POST".equalsIgnoreCase(t.getRequestMethod())) {
            String body = new String(t.getRequestBody().readAllBytes());
            int count = Integer.parseInt(extractJsonField(body, "count"));
            String[] candidates = {"Malla Reddy", "M.K. Stalin", "Pawan Kalyan",
"Balayya Babu", "Yogi Bhai", "Mamata Didi", "Jagan Reddy", "Rahul Gandhi", "RaviKishan",
"VD Sateeshan", "NOTA"};
            Random rand = new Random();
            for (int i = 0; i < count; i++) {
                String voterId = "Auto_" + System.currentTimeMillis() + "_" + i;
                String chosenCandidate =
candidates[rand.nextInt(candidates.length)];
                hasVotedSet.add(voterId);
                currentBatch.add(voterId + ":" + chosenCandidate);
            }
            sendJsonResponse(t, 200, "{\"status\":\"success\",\"message\":\"\" +
count + " votes added to buffer.\"}");
        }
    }

    static class SealHandler implements HttpHandler {
        @Override
        public void handle(HttpExchange t) throws IOException {
            if (currentBatch.isEmpty()) {
                sendJsonResponse(t, 400, "{\"status\":\"error\",\"message\":\"Batch
buffer is empty.\"}");
                return;
            }
            Block newBlock = new Block(currentBatch,
electionChain.getLatestBlock().hash);
            electionChain.addBlock(newBlock);
            currentBatch.clear();
            sendJsonResponse(t, 200, "{\"status\":\"success\",\"message\":\"Block
sealed!\"}");
        }
    }

    static class AuditHandler implements HttpHandler {
        @Override
        public void handle(HttpExchange t) throws IOException {
            boolean valid = true;
            int brokenIndex = -1;
            for (int i = 1; i < electionChain.chain.size(); i++) {
                Block current = electionChain.chain.get(i);

```



```

        Block previous = electionChain.chain.get(i - 1);
        MerkleTree checkTree = new MerkleTree(current.votes);
        if (!current.merkleRoot.equals(checkTree.getRoot()) ||
            !current.hash.equals(current.calculateBlockHash()) ||
            !current.previousHash.equals(previous.hash)) {
            valid = false;
            brokenIndex = i;
            break;
        }
    }
    sendJsonResponse(t, 200, "{\"isValid\":\"" + valid + "\",\"brokenBlockIndex\":\""
+ brokenIndex + "\"}");
}

static class TamperHandler implements HttpHandler {
    @Override
    public void handle(HttpExchange t) throws IOException {
        if ("POST".equalsIgnoreCase(t.getRequestMethod())) {
            String body = new String(t.getRequestBody().readAllBytes());
            int bIndex = Integer.parseInt(extractJsonField(body, "blockIndex"));
            int vIndex = Integer.parseInt(extractJsonField(body, "voteIndex"));
            String fakeData = extractJsonField(body, "fakeData");
            if (bIndex <= 0 || bIndex >= electionChain.chain.size() || vIndex < 0 ||
vIndex >= electionChain.chain.get(bIndex).votes.size()) {
                sendJsonResponse(t, 400, "{\"status\":\"error\", \"message\":\"Index
out of bounds.\"}");
                return;
            }
            electionChain.chain.get(bIndex).votes.set(vIndex, fakeData);
            sendJsonResponse(t, 200,
"{\"status\":\"partial_success\", \"message\":\"Data overwritten, Merkle Root forgery
failed.\"}");
        }
    }

    static class ResetHandler implements HttpHandler {
        @Override
        public void handle(HttpExchange t) throws IOException {
            t.getRequestBody().readAllBytes();
            electionChain = new Blockchain();
            currentBatch.clear();
            hasVotedSet.clear();
            sendJsonResponse(t, 200, "{\"status\":\"success\", \"message\":\"Blockchain
reset.\"}");
        }
    }
}

```

13.3 Legacy Terminal Interface (VotingSystem.java & VotingSystem1.java)

These files contain the original command-line interface (CLI) implementations used for early-stage testing and manual block-sealing validation prior to the web dashboard integration.

VotingSystem.java:



Java

```
import java.util.*;
public class VotingSystem {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        Blockchain electionChain = new Blockchain();
        List<String> currentBatch = new ArrayList<>();
        Set<String> hasVotedSet = new HashSet<>();
        String[] candidates = {
            "Sosuke Aizen",
            "Makima",
            "Johan Libert",
            "Light Yagami",
            "L Lawliet"
        };
        boolean isVotingOpen = true;
        System.out.println("=====");
        System.out.println(" DECENTRALIZED ELECTION TERMINAL ");
        System.out.println("=====");
        while (isVotingOpen) {
            System.out.println("\n--- MAIN MENU ---");
            System.out.println("1. Cast a Vote");
            System.out.println("2. Seal Current Batch into Blockchain");
            System.out.println("3. Verify Election Integrity");
            System.out.println("4. HACKER MODE: Tamper with a Block");
            System.out.println("5. End Election & Exit");
            System.out.print("Choose an option (1-5): ");
            String choice = scanner.nextLine();
            switch (choice) {
                case "1":
                    System.out.print("Enter Voter ID: ");
                    String voterId = scanner.nextLine();
                    if (hasVotedSet.contains(voterId)) {
                        System.out.println(">> SECURITY ALERT: Vote Rejected! Voter ID '" + voterId + "' has already cast a vote.");
                        break;
                    }
                    System.out.println("\nCandidates:");
                    for (int i = 0; i < candidates.length; i++) {
                        System.out.println((i + 1) + ". " + candidates[i]);
                    }
                    System.out.print("Select a candidate (1-5): ");
                    int candChoice;
                    try {
                        candChoice = Integer.parseInt(scanner.nextLine());
                        if (candChoice < 1 || candChoice > 5) throw new Exception();
                    } catch (Exception e) {
                        System.out.println(">> Invalid candidate selection. Vote cancelled.");
                        break;
                    }
                    String candidate = candidates[candChoice - 1];
                    hasVotedSet.add(voterId);
                    String voteData = voterId + ":" + candidate;
                    currentBatch.add(voteData);
                    System.out.println(">> Vote securely buffered for " + candidate + "!" + (" + currentBatch.size() + " vote(s) waiting to be sealed");
```



```
break;

case "2":
    if (currentBatch.isEmpty()) {
        System.out.println(">> No votes in the current batch to seal.");
    } else {
        System.out.println(">> Sealing batch of " + currentBatch.size()
+ " vote(s) into the blockchain...");
        Block newBlock = new Block(currentBatch,
electionChain.getLatestBlock().hash);
        electionChain.addBlock(newBlock);

        currentBatch.clear();
        System.out.println(">> Block successfully added! Merkle Root
recorded.");
    }
    break;

case "3":
    System.out.println(">> Running cryptographic audit on all
blocks...");
    if (electionChain.isChainValid()) {
        System.out.println(">> STATUS: SECURE. No tampering detected.");
    } else {
        System.out.println(">> STATUS: ALERT! Election data has been
tampered with!");
    }
    break;

case "4":
    if (electionChain.chain.size() <= 1) {
        System.out.println(">> No sealed blocks available to tamper with
yet! Seal a batch first.");
        break;
    }

    System.out.println("\n--- HACKER TERMINAL ---");
    System.out.println("Available blocks to hack: 1 to " +
(electionChain.chain.size() - 1));
    System.out.print("Select Block Number to alter: ");

    int blockIndex;
    try {
        blockIndex = Integer.parseInt(scanner.nextLine());
        if (blockIndex < 1 || blockIndex >= electionChain.chain.size())
throw new Exception();
    } catch (Exception e) {
        System.out.println(">> Invalid block selection.");
        break;
    }
    Block hackedBlock = electionChain.chain.get(blockIndex);
    System.out.println("Votes in Block " + blockIndex + ":");
    for (int i = 0; i < hackedBlock.votes.size(); i++) {
        System.out.println(i + ": " + hackedBlock.votes.get(i));
    }
    System.out.print("Select Vote Index to alter (0 to " +
(hackedBlock.votes.size() - 1) + "): ");
    int voteIndex;
    try {
```



```

        voteIndex = Integer.parseInt(scanner.nextLine());
        if (voteIndex < 0 || voteIndex >= hackedBlock.votes.size())
throw new Exception();
        } catch (Exception e) {
            System.out.println(">> Invalid vote index.");
            break;
        }
        System.out.print("Enter the new tampered vote data (e.g.,
Hacker_ID:Candidate_X): ");
        String fakeData = scanner.nextLine();
        hackedBlock.votes.set(voteIndex, fakeData);
        MerkleTree tamperedTree = new MerkleTree(hackedBlock.votes);
        hackedBlock.merkleRoot = tamperedTree.getRoot();
        System.out.println(">> HACK SUCCESSFUL: Data overwritten and Merkle
Root recalculated.");
        System.out.println(">> Run 'Verify Election Integrity' (Option 3) to
see if the Blockchain catches it!");
        break;
    case "5":
        System.out.println(">> Shutting down election system...");
        if (!currentBatch.isEmpty()) {
            System.out.println(">> Warning: " + currentBatch.size() + "
unsealed vote(s) were discarded.");
        }
        isVotingOpen = false;
        break;
    default:
        System.out.println(">> Invalid option. Please select 1-5.");
    }
}
}
scanner.close();
}
}

```

VotingSystem1.java:

```

import java.util.*;
public class VotingSystem1 {
    public static void main(String[] args) throws InterruptedException {
        Blockchain electionChain = new Blockchain();
        List<String> currentBatch = new ArrayList<>();
        System.out.println("=====\\n
DECENTRALIZED ELECTION: AUTOMATED RUN
\\n=====\\n");
        List<String> predefinedVotes = new ArrayList<>();
        for (int i = 0; i < 15; i++) predefinedVotes.add("Malla Reddy");
        for (int i = 0; i < 12; i++) predefinedVotes.add("Pawan Kalyan");
        for (int i = 0; i < 10; i++) predefinedVotes.add("Balayya Babu");
        for (int i = 0; i < 8; i++) predefinedVotes.add("M.K. Stalin");
        for (int i = 0; i < 5; i++) predefinedVotes.add("Jagan Babu");
        for (int i = 0; i < 4; i++) predefinedVotes.add("Rahul Gandhi");
        for (int i = 0; i < 2; i++) predefinedVotes.add("Yogi Bhai");
        for (int i = 0; i < 2; i++) predefinedVotes.add("Mamata Didi");
        for (int i = 0; i < 1; i++) predefinedVotes.add("RaviKishan");
    }
}

```



```
for (int i = 0; i < 1; i++) predefinedVotes.add("VD Sateeshan");
Collections.shuffle(predefinedVotes);
int blockCounter = 1;
System.out.println(">> Processing 60 voters into blocks of 10...\n");
for (int i = 0; i < 60; i++) {
    String voterId = "Voter_" + String.format("%03d", (i + 1));
    String candidate = predefinedVotes.get(i);
    currentBatch.add(voterId + ":" + candidate);
    if (currentBatch.size() == 10) {
        Block newBlock = new Block(currentBatch,
electionChain.getLatestBlock().hash);
        electionChain.addBlock(newBlock);
        System.out.println("--- Block " + blockCounter + " Sealed ---
\nVotes included: 10\nMerkle Root: " + newBlock.merkleRoot + "\nBlock Hash: " +
newBlock.hash + "\n");
        currentBatch.clear();
        blockCounter++;
    }
}
Scanner scanner = new Scanner(System.in);
System.out.println("===== \n HACKER
TERMINAL ACTIVATED \n=====");
System.out.print("Do you want to attempt to tamper with the election
data? (Y/N): ");
String hackChoice = scanner.nextLine();
if (hackChoice.equalsIgnoreCase("Y")) {
    System.out.println("\nAvailable blocks to hack: 1 through 6");
    System.out.print("Select Block Number to alter: ");
    int blockIndex = Integer.parseInt(scanner.nextLine());
    if (blockIndex < 1 || blockIndex > 6) {
        System.out.println(">> Invalid Block. Terminating hacker
tool.");
        scanner.close();
        return;
    }
    Block hackedBlock = electionChain.chain.get(blockIndex);
    System.out.println("\nVotes in Block " + blockIndex + ":");
    for (int i = 0; i < hackedBlock.votes.size(); i++)
System.out.println(i + ": " + hackedBlock.votes.get(i));
    System.out.print("\nSelect Vote Index to alter (0-9): ");
    int voteIndex = Integer.parseInt(scanner.nextLine());
    System.out.print("Enter the forged vote data: ");
    String fakeData = scanner.nextLine();
    Thread.sleep(1000);
    hackedBlock.votes.set(voteIndex, fakeData);
    Thread.sleep(1500);
    MerkleTree tamperedTree = new MerkleTree(hackedBlock.votes);
    hackedBlock.merkleRoot = tamperedTree.getRoot();
    Thread.sleep(1000);
    System.out.println("\n--- RUNNING SYSTEM-WIDE CRYPTOGRAPHIC AUDIT --
-");
    Thread.sleep(1500);
    if (electionChain.isChainValid()) {
```



```

        System.out.println(">> STATUS: Hack successful. System failed to
detect tampering.");
    } else {
        System.out.println(">> STATUS: [CRITICAL ALERT] INTEGRITY
COMPROMISED!\n>> RESULT: Blockchain validation failed. Attack isolated and
exposed.");
    }
    } else {
        System.out.println(">> Election secured safely. Shutting down.");
    }
    System.out.println("=====\n OFFICIAL
ELECTION RESULTS \n=====");
    Map<String, Integer> voteTally = new HashMap<>();
    for (int i = 1; i < electionChain.chain.size(); i++) {
        Block block = electionChain.chain.get(i);
        for (String voteData : block.votes) {
            String candidate = voteData.split(":")[1];
            voteTally.put(candidate, voteTally.getDefault(candidate, 0) +
1);
        }
    }
    String declaredWinner = "";
    int highestVotes = 0;
    boolean isTie = false;
    for (Map.Entry<String, Integer> entry : voteTally.entrySet()) {
        System.out.println("- " + entry.getKey() + ": \t" + entry.getValue()
+ " votes");
        if (entry.getValue() > highestVotes) {
            highestVotes = entry.getValue();
            declaredWinner = entry.getKey();
            isTie = false;
        } else if (entry.getValue() == highestVotes) {
            isTie = true;
        }
    }
    if (isTie) {
        System.out.println("\n*** ELECTION RESULT: TIE - NO OUTRIGHT WINNER
***\n");
    } else {
        System.out.println("\n*** WINNER DECLARED: " + declaredWinner + "
with " + highestVotes + " votes! ***\n");
    }
    scanner.close();
}
}

```